

Functions in Rust

Functions are declared with the keyword *fn*. An example of a function in Rust is given below:

```
fn main() {
    println!("Hello, world!");
}
```

When passing arguments to a function, it is imperative that the data types of the arguments are declared, as shown in the example below:

```
fn print_sum_of_numbers(num1: i8, num2: i8) -> i32
{
    println!("addition is: {}", num1 + num2);
    num1 + num2 //no ; means an expression, return num1+num2
}
```

In Line 1 of the above snippet, `-> i32` determines the return type of the value returned by the function `print_sum`. In Rust, the function body can be assigned to a variable, described as function pointers, as shown in the snippet below:

```
// Function pointers
fn pointerExample(count: i32) -> i32 {
    count + 1 //no ; means an expression, return count +1
}

let count1= pointerExample;
let count2 = count1(5); //6
```

Control flows in Rust

A wide array of control flows is available in Rust, including popular ones like *if-else*, *for* and *while* loops, as well as some control flows exclusive to Rust, i.e., *match* and *loop*. An example of these is shown in the code snippet below:

```
// if-else Example
let comp_size = 7;
if comp_size < 5 {
    println!("Small Company");
} else if comp_size < 10 {
    println!("Medium Company");
} else {
    println!("Large Company");
}

// while loop example

let mut count= 0;
while count < 5 {
    if count == 0 {
        println!("Omit value : {}", b count
```

```
        count += 1;
        continue;
    } else if count == 2 {
        println!("exit At : {}", count);
        break;
    }
    println!("value : {}", count);
    count += 1;
}
```

```
// for Loop Example
for count in 0..10 {
    if count == 0 {
        println!("Omit Value : {}", count);
        continue;
    } else if count == 2 {
        println!("Exit At : {}", count);
        break;
    }
    println!("value : {}", count);
}
```

Another unique control flow available in Rust is *loop*, which is used to indicate an infinite loop, out-of-the-box. The *break* statement can be used to exit a loop at any time, whereas the *continue* statement can be used to skip the rest of the iteration and start a new one, as shown in the code snippet below:

```
fn main() {
    let mut count = 0u32;

    println!("Let's count until infinity!");

    // Infinite loop
    loop {
        count += 1;

        if count == 3 {
            println!("three");

            // Skip the rest of this iteration
            continue;
        }

        println!("{}", count);

        if count == 5 {
            println!("OK, that's enough");

            // Exit this loop
            break;
        }
    }
}
```